

Consulta: Metodologías Ágiles

Scrum, Extreme Programming (XP), Kanban y Crystal



**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**

Autores:

Dania Lizeth Guzmán Triviño - 20221020061

Juan Sebastián Vega Díaz - 20231020087

Johan Felipe Pinzon Garcia - 20222020176

Asignatura:

**FUNDAMENTOS DE INGENIERÍA DE
SOFTWARE**

Bogotá D.C., Colombia

2026

Índice

1. Introducción	3
2. Scrum	3
2.1. ¿Qué es Scrum?	3
2.2. ¿Cómo funciona?	3
2.3. Responsabilidades dentro del Scrum Team	4
2.4. Artefactos	4
2.5. Ventajas	4
2.6. Limitaciones	5
2.7. ¿Cuándo utilizar Scrum?	5
3. Extreme Programming (XP)	5
3.1. ¿Qué es XP?	5
3.2. Valores de XP	5
3.3. Principales prácticas	6
3.4. Prácticas técnicas importantes	6
3.4.1. Pair Programming	6
3.4.2. Test-Driven Development (TDD)	7
3.4.3. Continuous Integration	7
3.4.4. Refactoring	7
3.4.5. Small Releases	7
3.5. Ventajas	7
3.6. Limitaciones	7
3.7. ¿Cuándo utilizar XP?	8
4. Kanban	8
4.1. ¿Qué es Kanban?	8
4.2. Prácticas generales	8
4.3. Trabajo en progreso (WIP)	9
4.4. Flujo continuo	9
4.5. Ventajas	9
4.6. Limitaciones	9
4.7. ¿Cuándo utilizar Kanban?	10
5. Crystal	10
5.1. ¿Qué es Crystal?	10
5.2. La familia Crystal	10
5.3. Crystal Clear	11

5.4. Filosofía	11
5.5. Ventajas	11
5.6. Limitaciones	12
5.7. ¿Cuándo utilizar Crystal?	12
6. Comparación de las cuatro metodologías	12
7. Diferencia conceptual	13
8. Conclusiones	14
9. Fuentes de consulta	14

1. Introducción

Las metodologías ágiles surgieron como una respuesta a las dificultades que presentaban algunos enfoques tradicionales de desarrollo de software, especialmente cuando los requisitos podían cambiar durante el proyecto.

Es importante aclarar que **Agil no corresponde a una única metodología**. El término hace referencia a un conjunto de valores y principios para orientar el desarrollo de productos y software. Sobre estos principios se han desarrollado diferentes marcos, métodos y metodologías, entre ellos Scrum, Extreme Programming (XP), Kanban y Crystal.

El objetivo de esta consulta es presentar una visión general de estas cuatro propuestas, identificando su origen, filosofía, funcionamiento, principales elementos, ventajas, limitaciones y contextos en los que pueden resultar apropiadas.

2. Scrum

2.1. ¿Qué es Scrum?

Scrum es un **framework ágil** utilizado para desarrollar y mantener productos complejos. Su funcionamiento se basa en ciclos iterativos e incrementales denominados **Sprints**.

Scrum busca generar valor mediante un proceso en el que el equipo inspecciona frecuentemente el producto y la forma de trabajo y se adapta cuando es necesario.

Sus tres pilares fundamentales son:

- **Transparencia:** el proceso y el trabajo deben ser visibles para quienes realizan y reciben el trabajo.
- **Inspección:** los artefactos y el progreso se revisan frecuentemente para detectar problemas.
- **Adaptación:** cuando se detectan desviaciones o problemas, se realizan ajustes.

2.2. ¿Cómo funciona?

El trabajo se organiza alrededor de un Sprint, que tiene una duración fija de un mes o menos. Durante el Sprint se desarrolla un incremento de producto potencialmente utilizable.

De manera simplificada, el ciclo puede representarse así:

Product Backlog → Sprint Planning → Sprint → Increment → Sprint Review → Sprint Retrospective

Durante el Sprint también se realiza el **Daily Scrum**, cuyo objetivo es inspeccionar el progreso hacia el Sprint Goal y adaptar el plan cuando sea necesario.

2.3. Responsabilidades dentro del Scrum Team

El Scrum Team está compuesto por:

- **Product Owner:** responsable de maximizar el valor del producto y de gestionar eficazmente el Product Backlog.
- **Scrum Master:** responsable de ayudar a establecer Scrum, facilitar su comprensión y ayudar a eliminar impedimentos.
- **Developers:** personas que crean el incremento de producto durante el Sprint.

2.4. Artefactos

Scrum define tres artefactos principales:

- **Product Backlog:** lista ordenada y emergente de lo que se necesita para mejorar el producto.
- **Sprint Backlog:** conjunto de elementos seleccionados para el Sprint junto con el plan para entregarlos.
- **Increment:** resultado concreto y utilizable generado durante el Sprint.

Cada artefacto tiene asociado un compromiso:

- Product Backlog → Product Goal.
- Sprint Backlog → Sprint Goal.
- Increment → Definition of Done.

2.5. Ventajas

- Facilita la adaptación a cambios.
- Permite entregar incrementos frecuentemente.
- Hace visible el progreso.
- Promueve la inspección y adaptación.

- Define responsabilidades claras.
- Favorece equipos autogestionados.

2.6. Limitaciones

- Puede convertirse en una implementación burocrática si se siguen reuniones y roles sin comprender su propósito.
- Requiere disciplina y compromiso del equipo.
- Puede presentar dificultades si el Product Owner no está realmente involucrado.
- Scrum por sí mismo no garantiza el éxito de un producto.

2.7. ¿Cuándo utilizar Scrum?

Puede resultar apropiado cuando el producto está evolucionando, existe incertidumbre en los requisitos, se necesita retroalimentación frecuente y existe un equipo dedicado capaz de producir incrementos regularmente.

Idea clave: Scrum se enfoca principalmente en **organizar y gestionar el trabajo mediante ciclos cortos de desarrollo**.

3. Extreme Programming (XP)

3.1. ¿Qué es XP?

Extreme Programming (XP) es una metodología ágil enfocada especialmente en las prácticas técnicas de desarrollo de software, la calidad del código, la retroalimentación rápida y la capacidad de responder a cambios frecuentes.

XP fue desarrollada principalmente por **Kent Beck** y adquirió gran reconocimiento a partir del proyecto Chrysler Comprehensive Compensation System (C3).

Una diferencia importante frente a Scrum es que XP profundiza mucho más en **cómo se desarrolla técnicamente el software**.

3.2. Valores de XP

Los valores tradicionalmente asociados con XP son:

- Comunicación.

- Simplicidad.
- Retroalimentación.
- Coraje.
- Respeto.

Estos valores orientan las prácticas y decisiones del equipo.

3.3. Principales prácticas

Entre las prácticas originales de XP se encuentran:

1. Planning Game.
2. Small Releases.
3. Metaphor.
4. Simple Design.
5. Testing.
6. Refactoring.
7. Pair Programming.
8. Collective Ownership.
9. Continuous Integration.
10. 40-hour Week.
11. On-site Customer.
12. Coding Standards.

3.4. Prácticas técnicas importantes

3.4.1. Pair Programming

Dos desarrolladores trabajan conjuntamente sobre el mismo código. Mientras uno escribe, el otro puede revisar, razonar sobre la solución y detectar problemas.

Esta práctica favorece el intercambio de conocimiento y puede mejorar la calidad del código.

3.4.2. Test-Driven Development (TDD)

El desarrollo guiado por pruebas sigue, de manera simplificada, el ciclo:

Prueba que falla → Código → Refactorización

El objetivo es construir el software acompañado por pruebas automatizadas desde etapas tempranas.

3.4.3. Continuous Integration

Los cambios se integran frecuentemente al código principal para detectar conflictos y errores de integración lo antes posible.

3.4.4. Refactoring

Consiste en mejorar la estructura interna del código sin modificar su comportamiento observable. Busca aumentar la legibilidad, mantenibilidad y simplicidad.

3.4.5. Small Releases

XP promueve entregas pequeñas y frecuentes de software funcional, permitiendo obtener retroalimentación rápidamente.

3.5. Ventajas

- Fuerte orientación hacia la calidad técnica.
- Detección temprana de errores.
- Feedback rápido.
- Promueve el intercambio de conocimiento.
- Facilita la adaptación a cambios.
- Favorece la mantenibilidad del código.

3.6. Limitaciones

- Requiere una disciplina técnica elevada.
- Algunas prácticas pueden resultar difíciles de adoptar.

- La participación frecuente del cliente puede ser complicada.
- Algunas prácticas pueden presentar dificultades al escalar a equipos muy grandes.

3.7. ¿Cuándo utilizar XP?

Es especialmente interesante cuando existe alta incertidumbre técnica, los requisitos cambian frecuentemente, la calidad del código es crítica y se dispone de un equipo pequeño o mediano.

Idea clave: XP se concentra en **cómo desarrollar software de alta calidad mediante prácticas técnicas intensivas**.

4. Kanban

4.1. ¿Qué es Kanban?

Kanban es un método para **gestionar y mejorar el flujo de trabajo**. Su elemento visual más conocido es el tablero Kanban, en el que los elementos de trabajo se representan y avanzan por diferentes etapas.

Un ejemplo sencillo sería:

Por hacer	En desarrollo	En pruebas	Terminado
Login	API usuarios	Login	Registro
Reportes	Carrito	—	API productos
Pagos	—	—	—

Sin embargo, Kanban no consiste únicamente en utilizar un tablero. Su propósito es mejorar el flujo mediante un conjunto de prácticas.

4.2. Prácticas generales

1. **Visualizar el trabajo.**
2. **Limitar el trabajo en progreso (WIP).**
3. **Gestionar el flujo.**
4. **Hacer explícitas las políticas.**
5. **Implementar ciclos de retroalimentación.**

6. Mejorar colaborativamente y evolucionar experimentalmente.

4.3. Trabajo en progreso (WIP)

WIP significa *Work In Progress*, es decir, trabajo que se encuentra actualmente en proceso.

Kanban puede establecer, por ejemplo:

“Máximo tres tareas pueden estar simultáneamente en desarrollo.”

El objetivo es evitar que el equipo inicie demasiadas tareas al mismo tiempo y favorecer que el trabajo se termine antes de comenzar nuevo trabajo.

Esto ayuda a identificar cuellos de botella y mejorar la velocidad y previsibilidad del flujo.

4.4. Flujo continuo

Una diferencia importante frente a Scrum es que Kanban **no requiere Sprints**. El trabajo puede avanzar de manera continua.

De forma simplificada:

Entrada → Desarrollo → Pruebas → Entrega

El equipo incorpora nuevo trabajo cuando existe capacidad disponible.

4.5. Ventajas

- Es altamente visual.
- Es flexible.
- Facilita la identificación de cuellos de botella.
- Permite limitar la multitarea mediante WIP.
- Puede introducirse progresivamente.
- Resulta útil para trabajo continuo.

4.6. Limitaciones

- Un tablero por sí solo no constituye una implementación completa de Kanban.
- Requiere disciplina para respetar los límites WIP.
- Puede ser difícil gestionar prioridades cuando existe una demanda excesiva.

- No impone una estructura de roles y eventos tan definida como Scrum.

4.7. ¿Cuándo utilizar Kanban?

Puede resultar apropiado para equipos que trabajan con solicitudes continuas, mantenimiento de software, corrección de errores, soporte técnico o entornos donde las prioridades cambian frecuentemente.

Idea clave: Kanban busca **optimizar el flujo de trabajo y limitar el trabajo en progreso**.

5. Crystal

5.1. ¿Qué es Crystal?

Crystal es una **familia de metodologías ágiles** desarrollada por **Alistair Cockburn**. Su característica principal es que no propone un proceso idéntico para todos los proyectos.

Crystal plantea que el proceso debe adaptarse al contexto, considerando factores como:

- Tamaño del equipo.
- Criticidad del sistema.
- Necesidades de comunicación.
- Riesgos.
- Contexto organizacional.

Por este motivo se habla de una familia de metodologías y no de una única receta.

5.2. La familia Crystal

Crystal utiliza diferentes variantes asociadas a colores, entre ellas:

- Crystal Clear.
- Crystal Yellow.
- Crystal Orange.
- Crystal Red.

Los colores representan diferentes niveles de peso y características del proceso según el tamaño y criticidad del proyecto.

No se trata simplemente de memorizar los colores. Lo fundamental es comprender el principio de que **el proceso debe adaptarse al contexto**.

5.3. Crystal Clear

Crystal Clear es una de las variantes más conocidas y está orientada principalmente a equipos pequeños.

Da especial importancia a:

- Comunicación directa.
- Entrega frecuente.
- Mejora y reflexión del equipo.
- Acceso a usuarios o clientes.
- Seguridad para expresar problemas.
- Un entorno técnico apropiado.

5.4. Filosofía

Crystal concede una importancia considerable a las personas y a la comunicación. La idea es que las capacidades del equipo y la calidad de la comunicación pueden tener un impacto significativo sobre el resultado del proyecto.

Por ello, el método no debe ser más pesado de lo necesario.

5.5. Ventajas

- Alta adaptabilidad.
- Considera el contexto real del proyecto.
- Da gran importancia a las personas.
- Evita imponer procesos innecesariamente pesados.
- Puede ser apropiado para equipos pequeños.

5.6. Limitaciones

- Es menos estructurado que Scrum.
- Puede resultar difícil para equipos que necesitan reglas completamente definidas.
- Requiere criterio para decidir cómo adaptar el proceso.
- Tiene menor adopción generalizada que Scrum o Kanban.

5.7. ¿Cuándo utilizar Crystal?

Puede ser apropiado cuando el equipo es relativamente pequeño, la comunicación es fundamental y existe la necesidad de adaptar el proceso a las características particulares del proyecto.

Idea clave: Crystal busca **adaptar el proceso al equipo, al proyecto y a su nivel de criticidad.**

6. Comparación de las cuatro metodologías

Las cuatro propuestas comparten principios relacionados con la adaptación, la entrega de valor y la mejora continua, pero tienen énfasis diferentes.

Característica	Scrum	XP	Kanban	Crystal
Enfoque principal	Gestión del trabajo y producto	Prácticas técnicas y calidad	Flujo de trabajo	Adaptación al contexto
Unidad de trabajo	Sprints	Iteraciones y pequeñas entregas	Flujo continuo	Iteraciones/ciclos
Roles definidos	Sí	Menos centrado en roles formales	No impone roles específicos	Adaptables
Tablero	Común	Puede utilizarse	Fundamental	Puede utilizarse
Límite WIP	No es central	No es central	Sí, es fundamental	Puede adaptarse
TDD	No obligatorio	Muy importante	No obligatorio	No obligatorio
Pair Programming	No obligatorio	Práctica importante	No obligatorio	Puede utilizarse
Sprints	Sí	Puede trabajar iterativamente	No obligatorios	No constituyen su elemento central
Adaptabilidad	Alta	Muy alta	Muy alta	Muy alta
Fortaleza principal	Organización	Calidad técnica	Flujo	Adaptación

7. Diferencia conceptual

Una manera sencilla de recordar las diferencias es:

Scrum: organizar el trabajo.

XP: desarrollar software con calidad técnica.

Kanban: hacer que el trabajo fluya.

Crystal: adaptar el proceso al contexto.

Esta diferencia también explica por qué algunas de estas propuestas pueden combinarse. Por ejemplo, un equipo puede utilizar Scrum para organizar el trabajo y adoptar prácticas técnicas de XP para mejorar la calidad del software.

8. Conclusiones

Las metodologías ágiles no deben entenderse como recetas universales. Todas parten de una filosofía que favorece la entrega de valor, la colaboración, la retroalimentación y la capacidad de adaptación.

Scrum destaca por proporcionar una estructura clara para organizar el trabajo en Sprints. XP profundiza en las prácticas técnicas necesarias para desarrollar software con calidad. Kanban se concentra en el flujo continuo y en limitar el trabajo en progreso. Crystal, por su parte, destaca por adaptar el proceso a las características del equipo y del proyecto.

Por lo tanto, seleccionar una metodología no consiste en preguntar simplemente cuál es la “mejor”, sino cuál resulta más apropiada para las condiciones particulares del proyecto.

Una pregunta fundamental para la ingeniería de software es:

¿Qué características tiene el proyecto, el equipo y el contexto, y qué enfoque permite gestionar mejor esas condiciones?

9. Fuentes de consulta

- Agile Manifesto. *Manifiesto para el Desarrollo Ágil de Software*. Disponible en: <https://agilemanifesto.org/iso/es/manifesto.html>
- Agile Manifesto. *Principios detrás del Manifiesto Ágil*. Disponible en: <https://agilemanifesto.org/iso/es/principles.html>
- Schwaber, K. & Sutherland, J. (2020). *The Scrum Guide*. Disponible en: <https://scrumguides.org/scrum-guide.html>
- Agile Alliance. *Extreme Programming (XP)*. Disponible en: <https://agilealliance.org/glossary/xp/>
- Kanban University. *The Official Guide to the Kanban Method*. Disponible en: <https://kanban.university/kanban-guide/>
- Cockburn, A. *Crystal Clear: A Human-Powered Methodology for Small Teams*. Addison-Wesley.